



Machine Learning

Learning Plan

From Mathematical Foundations to Research-Level Expertise

A Comprehensive, Structured Roadmap
Covering Classical ML → Statistical Learning → Research

Prepared for **Debadrito**

October 2026

Deep Learning & Neural Networks covered in Volume II

Contents

1	Overview & Philosophy	2
1.1	How to Use This Plan	2
1.2	Roadmap at a Glance	3
2	Phase 1: Mathematical Foundations	4
2.1	Linear Algebra	4
2.1.1	Core Topics (Must Master)	4
2.1.2	Practice Exercises	5
2.2	Multivariate Calculus	5
2.2.1	Core Topics	5
2.3	Optimisation Theory	6
2.3.1	Core Topics	6
3	Phase 2: Probability, Statistics & Information Theory	9
3.1	Probability Theory	9
3.1.1	Core Topics	9
3.2	Statistical Inference	9
3.2.1	Frequentist Methods	10
3.2.2	Bayesian Methods	10
3.3	Information Theory	10
3.3.1	Core Topics	10
4	Phase 3: Core Machine Learning Algorithms	12
4.1	Supervised Learning — Regression	12
4.1.1	Linear Regression	12
4.1.2	Advanced Regression Topics	12
4.2	Supervised Learning — Classification	13
4.2.1	Logistic Regression	13
4.2.2	Support Vector Machines (SVM)	13
4.2.3	Decision Trees	14
4.2.4	Naive Bayes Classifiers	14
4.2.5	k -Nearest Neighbours (kNN)	14
4.3	Ensemble Methods	14
4.4	Unsupervised Learning	15
4.4.1	Clustering	15
4.4.2	Dimensionality Reduction	16
4.5	Model Evaluation & Selection	16
5	Phase 4: Advanced ML & Statistical Learning Theory	18
5.1	Statistical Learning Theory	18
5.2	Bayesian Machine Learning (Advanced)	19
5.3	Graphical Models	19
5.4	Advanced Topics in Classical ML	20

6	Phase 5: ML Systems, Engineering & Scalability	22
6.1	Data Engineering for ML	22
6.2	ML Pipeline Design	22
6.3	Scalable ML	23
7	Phase 6: Research Methodology & Paper Writing	24
7.1	Reading Research Papers	24
7.2	Designing Experiments	24
7.3	Writing ML Papers	25
7.4	Research Project Execution	26
8	Appendices	27
8.1	Appendix A: Recommended Book Progression	27
8.2	Appendix B: Weekly Study Template	27
8.3	Appendix C: Python Environment Setup	28
8.4	Appendix D: Key Mathematical Notation	28

1 Overview & Philosophy

Learning Philosophy

This plan is designed to take you from **zero to research-level competency** in classical Machine Learning. It deliberately excludes Deep Learning and Neural Networks (which will be covered in a separate Volume II) so that you build an **unshakeable foundation** in the mathematics, statistics, and algorithmic thinking that underpins all of modern ML.

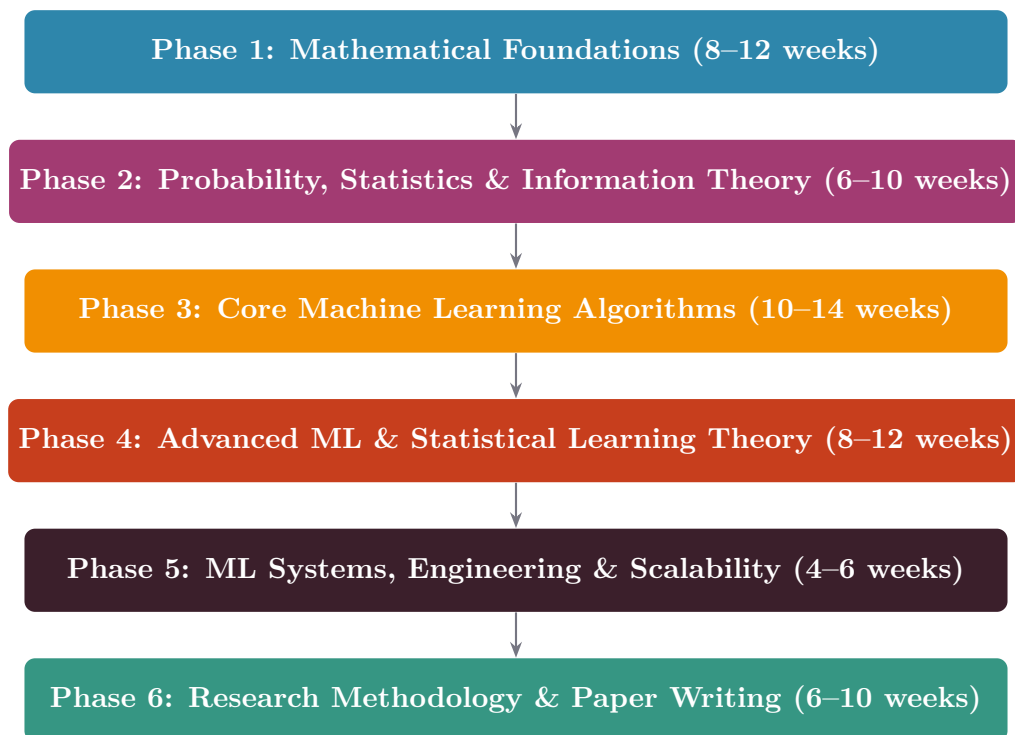
Why this ordering matters:

- Research papers assume fluency in linear algebra, probability, and optimisation.
- Understanding classical methods deeply makes DL/NN intuitions *much* clearer.
- Many real-world problems are still best solved by classical ML.
- Reviewers and collaborators expect mathematical rigour at the research level.

1.1 How to Use This Plan

1. **Follow the phases sequentially.** Each phase builds on the previous one.
2. **Do every exercise and project.** Reading without coding builds false confidence.
3. **Read the referenced papers.** Summaries are no substitute for primary sources.
4. **Keep a learning journal.** Write proofs, derivations, and code walkthroughs.
5. **Timelines are estimates.** Adjust based on your background; rigour > speed.

1.2 Roadmap at a Glance



Total estimated duration: 42–64 weeks (10–16 months) at 15–20 hrs/week.

2 Phase 1: Mathematical Foundations

Duration: 8–12 weeks

Difficulty: Moderate to Hard

Goal: Develop fluency in the mathematical language of ML so that you can read, write, and derive equations found in research papers without hesitation.

2.1 Linear Algebra

Duration: 3–4 weeks

2.1.1 Core Topics (Must Master)

1. Vectors & Vector Spaces

- Definition of a vector space, subspaces, span, linear independence
- Basis, dimension, change of basis
- Inner products, norms ($\ell_1, \ell_2, \ell_\infty$), orthogonality
- Gram–Schmidt orthogonalisation process

2. Matrices & Linear Maps

- Matrix operations: addition, multiplication, transpose, inverse
- Rank, null space (kernel), column space, row space
- Determinants — geometric interpretation and computation
- Trace and its properties
- Special matrices: symmetric, orthogonal, positive (semi-)definite, diagonal, sparse

3. Systems of Linear Equations

- Gaussian elimination, LU decomposition
- Existence and uniqueness of solutions
- Over-determined and under-determined systems
- Least squares solution and the normal equations: $\hat{\mathbf{x}} = (\mathbf{A}^T \mathbf{A})^{-1} \mathbf{A}^T \mathbf{b}$

4. Eigendecomposition

- Eigenvalues and eigenvectors: definition, computation, geometric meaning
- Diagonalisation: $\mathbf{A} = \mathbf{P} \mathbf{\Lambda} \mathbf{P}^{-1}$
- Spectral theorem for symmetric matrices
- Positive definite matrices and their role in ML (covariance, kernels)

5. Singular Value Decomposition (SVD)

- Full derivation: $\mathbf{A} = \mathbf{U} \mathbf{\Sigma} \mathbf{V}^T$
- Relationship to eigendecomposition

- Low-rank approximation and the Eckart–Young theorem
- Applications: PCA, pseudoinverse, matrix completion, LSA

Key Identity to Internalise: For any matrix $\mathbf{A} \in \mathbb{R}^{m \times n}$, the SVD gives the best rank- k approximation in both Frobenius and spectral norms:

$$\min_{\text{rank}(\mathbf{B})=k} \|\mathbf{A} - \mathbf{B}\|_F = \sqrt{\sigma_{k+1}^2 + \cdots + \sigma_{\min(m,n)}^2}$$

2.1.2 Practice Exercises

- Implement SVD from scratch using the power method in Python (NumPy only).
- Prove that eigenvectors of a real symmetric matrix with distinct eigenvalues are orthogonal.
- Use SVD to compress an image and plot reconstruction error vs. number of singular values.
- Derive the pseudoinverse $\mathbf{A}^+ = \mathbf{V}\mathbf{\Sigma}^+\mathbf{U}^T$ and verify numerically.

Resources & References

- **Primary:** *Introduction to Linear Algebra* — Gilbert Strang (MIT OCW 18.06)
- **Deep Dive:** *Linear Algebra Done Right* — Sheldon Axler
- **ML-Focused:** *Mathematics for Machine Learning* — Deisenroth, Faisal, Ong (Ch. 2–4)
- **Video:** 3Blue1Brown — *Essence of Linear Algebra* (YouTube)
- **Coding:** NumPy documentation & exercises

2.2 Multivariate Calculus

Duration: 2–3 weeks

2.2.1 Core Topics

1. Differentiation in Multiple Variables

- Partial derivatives, gradient vector ∇f
- Directional derivatives and their geometric meaning
- The Jacobian matrix $\mathbf{J} \in \mathbb{R}^{m \times n}$ for vector-valued functions
- The Hessian matrix $\mathbf{H} \in \mathbb{R}^{n \times n}$ and second-order information
- Chain rule for compositions of multivariate functions

2. Matrix Calculus

- Derivatives of scalar functions w.r.t. vectors and matrices

- Key identities: $\frac{\partial}{\partial \mathbf{x}}(\mathbf{x}^T \mathbf{A} \mathbf{x}) = (\mathbf{A} + \mathbf{A}^T) \mathbf{x}$
- Derivative of the log-determinant: $\frac{\partial}{\partial \mathbf{A}} \log |\mathbf{A}| = \mathbf{A}^{-T}$
- The Matrix Cookbook as a reference

3. Taylor Series Approximation

- First-order (linear) approximation
- Second-order (quadratic) approximation using the Hessian
- Connection to Newton's method in optimisation

4. Integration (Selected Topics)

- Multivariable integration for normalisation constants
- Gaussian integrals: $\int_{-\infty}^{\infty} e^{-x^2} dx = \sqrt{\pi}$
- Change of variables formula (Jacobian determinant)

Pro Tip

Master matrix calculus notation early. Every ML paper uses gradients w.r.t. parameter vectors/matrices. The denominator-layout vs. numerator-layout distinction trips up many learners — pick one convention and stick with it.

Resources & References

- **Primary:** *Mathematics for Machine Learning* — Deisenroth et al. (Ch. 5)
- **Reference:** *The Matrix Cookbook* — Petersen & Pedersen (free PDF)
- **Video:** Khan Academy Multivariable Calculus; 3Blue1Brown Calculus series
- **Practice:** MIT OCW 18.02 problem sets

2.3 Optimisation Theory

Duration: 3–4 weeks

2.3.1 Core Topics

1. Unconstrained Optimisation

- First-order necessary conditions: $\nabla f(\mathbf{x}^*) = \mathbf{0}$
- Second-order sufficient conditions: $\mathbf{H}(\mathbf{x}^*) \succ 0$
- Convexity: convex sets, convex functions, strict/strong convexity
- Global vs. local minima; saddle points

2. Gradient-Based Methods

- Gradient descent: $\mathbf{x}_{t+1} = \mathbf{x}_t - \eta \nabla f(\mathbf{x}_t)$
- Convergence rates: $O(1/t)$ for convex, $O(\rho^t)$ for strongly convex ($\rho < 1$)

- Step size selection: fixed, decaying, line search (Armijo, Wolfe conditions)
- Stochastic Gradient Descent (SGD) and mini-batch SGD
- Momentum methods: Polyak momentum, Nesterov accelerated gradient
- Adaptive methods: AdaGrad, RMSProp, Adam (derive update rules)

3. Second-Order Methods

- Newton's method: $\mathbf{x}_{t+1} = \mathbf{x}_t - \mathbf{H}^{-1} \nabla f(\mathbf{x}_t)$
- Quasi-Newton methods: BFGS, L-BFGS
- When to prefer first-order vs. second-order methods

4. Constrained Optimisation

- Lagrange multipliers for equality constraints
- KKT conditions for inequality constraints
- Duality: weak and strong duality, Slater's condition
- Applications to SVM, constrained regression

5. Convex Optimisation

- Convex problems: LP, QP, SOCP, SDP
- Proximal operators and proximal gradient descent
- ADMM (Alternating Direction Method of Multipliers) — overview

The Central Equation of ML:

$$\boldsymbol{\theta}^* = \arg \min_{\boldsymbol{\theta}} \frac{1}{N} \sum_{i=1}^N \mathcal{L}(f(\mathbf{x}_i; \boldsymbol{\theta}), y_i) + \lambda \Omega(\boldsymbol{\theta})$$

Every ML algorithm is an instance of this template. Understanding optimisation means understanding how to solve this efficiently and correctly.

Hands-On Projects

1. Implement gradient descent, SGD, Adam from scratch; compare convergence on Rosenbrock's function.
2. Solve a constrained optimisation problem using KKT conditions by hand, then verify with `scipy.optimize`.
3. Visualise gradient descent trajectories on 2D loss surfaces with different learning rates.

Resources & References

- **Primary:** *Convex Optimization* — Boyd & Vandenberghe (free online, Stanford EE364a)
- **ML-Focused:** *Optimization for Machine Learning* — Sra, Nowozin, Wright

- **Practical:** Sebastian Ruder — “An Overview of Gradient Descent Optimization Algorithms” (2016)

3 Phase 2: Probability, Statistics & Information Theory

Duration: 6–10 weeks

Difficulty: Hard

Goal: Think probabilistically. Model uncertainty. Understand the statistical foundations that make ML rigorous and principled.

3.1 Probability Theory

Duration: 2–3 weeks

3.1.1 Core Topics

1. Foundations

- Sample spaces, events, probability axioms (Kolmogorov)
- Conditional probability, Bayes' theorem, independence
- Law of total probability, chain rule of probability

2. Random Variables & Distributions

- Discrete: Bernoulli, Binomial, Poisson, Categorical, Multinomial
- Continuous: Uniform, Gaussian (univariate & multivariate), Exponential, Beta, Gamma, Dirichlet
- PDF, PMF, CDF; expectation, variance, covariance, correlation
- Moment generating functions and characteristic functions
- The multivariate Gaussian: $\mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma})$ — marginals, conditionals, Schur complement

3. Important Theorems

- Law of Large Numbers (weak and strong)
- Central Limit Theorem and its implications
- Concentration inequalities: Markov, Chebyshev, Hoeffding, Chernoff

4. Joint & Conditional Distributions

- Marginalisation: $p(x) = \int p(x, y) dy$
- Bayes' rule in continuous form: $p(\theta|D) = \frac{p(D|\theta)p(\theta)}{p(D)}$
- Conjugate priors (Beta-Binomial, Normal-Normal, Dirichlet-Multinomial)

3.2 Statistical Inference

Duration: 2–3 weeks

3.2.1 Frequentist Methods

1. Point Estimation

- Maximum Likelihood Estimation (MLE): $\hat{\theta}_{\text{MLE}} = \arg \max_{\theta} \prod_{i=1}^N p(x_i|\theta)$
- Properties: consistency, asymptotic normality, efficiency
- Method of Moments

2. Confidence Intervals & Hypothesis Testing

- Construction of confidence intervals
- t-tests, χ^2 -tests, F-tests
- p-values, Type I/II errors, power analysis
- Multiple hypothesis testing: Bonferroni, Benjamini-Hochberg (FDR)

3. Bias-Variance Trade-off

- Decomposition: $\text{MSE} = \text{Bias}^2 + \text{Variance} + \text{Irreducible Error}$
- Implications for model selection and regularisation

3.2.2 Bayesian Methods

1. Bayesian Inference Framework

- Prior, likelihood, posterior, evidence (marginal likelihood)
- Maximum A Posteriori (MAP) estimation
- Connection: MAP with Gaussian prior $\equiv \ell_2$ -regularised MLE

2. Approximate Inference (Overview)

- Why exact inference is often intractable
- Laplace approximation
- Markov Chain Monte Carlo (MCMC): Metropolis-Hastings, Gibbs sampling
- Variational inference: ELBO, mean-field approximation

3.3 Information Theory

Duration: 2–3 weeks

3.3.1 Core Topics

1. Entropy & Information

- Shannon entropy: $H(X) = -\sum_x p(x) \log p(x)$
- Differential entropy for continuous distributions
- Joint entropy, conditional entropy, chain rule

2. Divergences & Distances

- KL divergence: $D_{\text{KL}}(p\|q) = \sum_x p(x) \log \frac{p(x)}{q(x)}$
- Properties: non-negative (Gibbs' inequality), asymmetric, not a metric
- Cross-entropy: $H(p, q) = H(p) + D_{\text{KL}}(p\|q)$
- Mutual information: $I(X; Y) = H(X) - H(X|Y)$
- Jensen-Shannon divergence, f-divergences (overview)

3. Applications to ML

- Cross-entropy loss in classification
- Mutual information for feature selection
- Information bottleneck principle
- Minimum description length (MDL) for model selection

Connecting the Dots:

$$\underbrace{-\log p(\mathbf{y}|\mathbf{X}, \boldsymbol{\theta})}_{\text{Negative Log-Likelihood}} = \underbrace{N \cdot H(p_{\text{data}}, p_{\theta})}_{\text{Cross-Entropy}} = \underbrace{N [H(p_{\text{data}}) + D_{\text{KL}}(p_{\text{data}}\|p_{\theta})]}_{\text{Entropy + KL Divergence}}$$

Minimising cross-entropy loss $\equiv$ maximising likelihood $\equiv$ minimising KL divergence from the data distribution.

Resources & References

- **Probability:** *Probability and Statistics for Computer Scientists* — Baron; or MIT OCW 6.041
- **Bayesian:** *Pattern Recognition and Machine Learning* — Bishop (Ch. 1–2); *Bayesian Reasoning and Machine Learning* — Barber
- **Information Theory:** *Elements of Information Theory* — Cover & Thomas (Ch. 1–4, 8–9)
- **Video:** StatQuest (YouTube) for intuitive statistics; mathematicalmonk for Bayesian ML

4 Phase 3: Core Machine Learning Algorithms

Duration: 10–14 weeks

Difficulty: Moderate to Hard

Goal: Understand, derive, implement from scratch, and critically evaluate every major classical ML algorithm. For each algorithm, know: (1) the model, (2) the loss function, (3) the optimisation procedure, (4) computational complexity, and (5) when to use it.

4.1 Supervised Learning — Regression

Duration: 2–3 weeks

4.1.1 Linear Regression

1. Ordinary Least Squares (OLS)

- Model: $y = \mathbf{x}^T \boldsymbol{\beta} + \varepsilon$
- Closed-form solution: $\hat{\boldsymbol{\beta}} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}$
- Geometric interpretation: projection onto column space
- Assumptions: linearity, homoscedasticity, independence, normality of residuals
- Gauss-Markov theorem (BLUE)

2. Ridge Regression (ℓ_2 Regularisation)

- $\hat{\boldsymbol{\beta}}_{\text{ridge}} = (\mathbf{X}^T \mathbf{X} + \lambda \mathbf{I})^{-1} \mathbf{X}^T \mathbf{y}$
- Bayesian interpretation as Gaussian prior on weights
- Effect on eigenvalues, bias-variance trade-off

3. LASSO (ℓ_1 Regularisation)

- Sparsity-inducing property: why ℓ_1 encourages zero coefficients
- Geometric interpretation: diamond vs. ball constraint sets
- Coordinate descent and LARS algorithms
- Elastic Net: combining ℓ_1 and ℓ_2

4. Polynomial & Basis Function Regression

- Feature expansion: polynomial, radial basis, Fourier
- Overfitting demonstration with increasing degree

4.1.2 Advanced Regression Topics

- Bayesian linear regression: predictive posterior, uncertainty quantification
- Gaussian Process Regression (connect to Phase 4)
- Robust regression: Huber loss, RANSAC

4.2 Supervised Learning — Classification

Duration: 3–4 weeks

4.2.1 Logistic Regression

1. Binary Classification

- Sigmoid function: $\sigma(z) = \frac{1}{1+e^{-z}}$
- Model: $p(y = 1|\mathbf{x}) = \sigma(\mathbf{w}^T \mathbf{x} + b)$
- Derive cross-entropy loss from MLE of Bernoulli likelihood
- Gradient computation and gradient descent
- Decision boundary geometry

2. Multinomial (Softmax) Regression

- Softmax function: $p(y = k|\mathbf{x}) = \frac{e^{\mathbf{w}_k^T \mathbf{x}}}{\sum_j e^{\mathbf{w}_j^T \mathbf{x}}}$
- Categorical cross-entropy loss
- One-vs-Rest (OvR) vs. One-vs-One (OvO) strategies

4.2.2 Support Vector Machines (SVM)

1. Hard-Margin SVM

- Maximum margin hyperplane: $\max_{\mathbf{w}, b} \frac{2}{\|\mathbf{w}\|}$ s.t. $y_i(\mathbf{w}^T \mathbf{x}_i + b) \geq 1$
- Support vectors and their role
- Primal and dual formulation (derive using Lagrangians)

2. Soft-Margin SVM

- Slack variables ξ_i , hinge loss: $\max(0, 1 - y_i f(\mathbf{x}_i))$
- Regularisation parameter C : trade-off between margin and violations

3. The Kernel Trick

- Mercer's theorem, positive definite kernels
- Common kernels: linear, polynomial, RBF (Gaussian), sigmoid
- Kernel matrix (Gram matrix) $K_{ij} = k(\mathbf{x}_i, \mathbf{x}_j)$
- Representer theorem
- Multi-class SVM strategies

4. Practical Considerations

- SMO (Sequential Minimal Optimisation) algorithm
- Feature scaling importance
- SVR (Support Vector Regression)

4.2.3 Decision Trees

1. Tree Construction

- Splitting criteria: Gini impurity, entropy (information gain), variance reduction
- Recursive partitioning (CART algorithm)
- Handling continuous and categorical features

2. Regularisation & Pruning

- Pre-pruning: max depth, min samples per leaf, min impurity decrease
- Post-pruning: cost-complexity pruning (α -pruning), reduced error pruning

4.2.4 Naive Bayes Classifiers

- Conditional independence assumption
- Gaussian, Multinomial, Bernoulli variants
- Laplace smoothing
- Why it works well despite “wrong” assumptions

4.2.5 k -Nearest Neighbours (kNN)

- Distance metrics: Euclidean, Manhattan, Minkowski, Mahalanobis
- Choice of k , curse of dimensionality
- KD-trees, Ball trees for efficient nearest-neighbour search
- Weighted kNN

4.3 Ensemble Methods

Duration: 2–3 weeks

1. Bagging (Bootstrap Aggregating)

- Bootstrap sampling, variance reduction
- Random Forests: feature subsampling, out-of-bag (OOB) error
- Feature importance: permutation importance, Gini importance

2. Boosting

- AdaBoost: weighted resampling, exponential loss
- Gradient Boosting Machines (GBM): functional gradient descent
- Derive GBM update for squared loss, logistic loss
- XGBoost: regularised objective, column sampling, sparsity-aware splits
- LightGBM: leaf-wise growth, histogram-based splitting

- CatBoost: ordered boosting, categorical feature handling

3. Stacking & Blending

- Meta-learner concept
- Cross-validated stacking to avoid leakage

Pro Tip

Gradient Boosted Decision Trees (GBDT) remain the most competitive classical ML method for tabular data. Master XGBoost/LightGBM thoroughly — they dominate Kaggle competitions and production systems.

4.4 Unsupervised Learning

Duration: 2–3 weeks

4.4.1 Clustering

1. k -Means

- Lloyd’s algorithm, convergence guarantees (to local optimum)
- Objective: $\min \sum_{k=1}^K \sum_{i \in C_k} \|\mathbf{x}_i - \boldsymbol{\mu}_k\|^2$
- Initialisation: k -means++
- Elbow method, silhouette score for choosing k

2. Gaussian Mixture Models (GMM)

- Model: $p(\mathbf{x}) = \sum_{k=1}^K \pi_k \mathcal{N}(\mathbf{x} | \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)$
- Expectation-Maximisation (EM) algorithm — full derivation
- E-step: compute responsibilities; M-step: update parameters
- Connection to k -means as a special case
- Model selection: BIC, AIC

3. Hierarchical Clustering

- Agglomerative: single, complete, average, Ward linkage
- Dendrograms and cutting strategies

4. Density-Based Clustering

- DBSCAN: core points, border points, noise; ϵ , minPts parameters
- HDBSCAN: hierarchical density-based clustering

5. Spectral Clustering

- Graph Laplacian: $\mathbf{L} = \mathbf{D} - \mathbf{W}$
- Normalised cuts, connection to eigenvector methods

4.4.2 Dimensionality Reduction

1. Principal Component Analysis (PCA)

- Derivation from maximum variance and minimum reconstruction error perspectives
- Connection to SVD and eigendecomposition of covariance matrix
- Choosing number of components: explained variance ratio, scree plot
- Kernel PCA for nonlinear dimensionality reduction
- Probabilistic PCA and Factor Analysis

2. Other Linear Methods

- Linear Discriminant Analysis (LDA) — supervised dimensionality reduction
- Independent Component Analysis (ICA)
- Non-negative Matrix Factorisation (NMF)

3. Manifold Learning

- t-SNE: Student- t distribution, perplexity, Barnes-Hut approximation
- UMAP: uniform manifold approximation, fuzzy simplicial sets
- Isomap, Locally Linear Embedding (LLE) — overview
- When to use t-SNE/UMAP vs. PCA

4.5 Model Evaluation & Selection

Duration: 1–2 weeks

1. Evaluation Metrics

- Classification: accuracy, precision, recall, F1, AUC-ROC, AUC-PR, log loss, confusion matrix, Cohen's κ
- Regression: MSE, RMSE, MAE, R^2 , adjusted R^2 , MAPE
- Ranking: NDCG, MAP
- Calibration: reliability diagrams, Brier score, Platt scaling, isotonic regression

2. Cross-Validation

- k -fold, stratified k -fold, leave-one-out (LOO)
- Nested cross-validation for unbiased hyperparameter tuning evaluation
- Time series cross-validation (expanding window, rolling window)
- Group cross-validation (GroupKFold)

3. Hyperparameter Optimisation

- Grid search, random search
- Bayesian optimisation: surrogate models (GP), acquisition functions (EI, UCB)

- Hyperband, successive halving
- Tools: Optuna, scikit-optimize

4. Model Selection Theory

- AIC, BIC, MDL
- Structural Risk Minimisation
- No Free Lunch theorem

Hands-On Projects

1. **From-Scratch Library:** Implement linear regression, logistic regression, SVM (using QP solver), decision tree, k -means, PCA, kNN in pure Python/NumPy. Write unit tests.
2. **Kaggle Tabular Competition:** Enter a tabular data competition. Use feature engineering, cross-validation, and gradient boosting. Aim for top 20%.
3. **Clustering Pipeline:** Take a real dataset (e.g., single-cell RNA-seq, customer segmentation). Apply PCA $\rightarrow$ UMAP $\rightarrow$ clustering. Evaluate with multiple metrics.
4. **Model Comparison Study:** On 5+ UCI datasets, compare all algorithms with proper cross-validation. Use statistical tests (Wilcoxon signed-rank) to compare.

Resources & References

- **Primary:** *The Elements of Statistical Learning (ESL)* — Hastie, Tibshirani, Friedman (free PDF)
- **Accessible:** *An Introduction to Statistical Learning (ISL)* — James, Witten, Hastie, Tibshirani (free PDF, with Python edition)
- **Comprehensive:** *Pattern Recognition and Machine Learning* — Bishop (“PRML”)
- **Practical:** *Hands-On Machine Learning* — Aurélien Géron (scikit-learn focus)
- **Code:** scikit-learn documentation and source code (excellent learning resource)
- **Video:** Andrew Ng’s Stanford CS229 (YouTube); Caltech Learning from Data (Abu-Mostafa)

5 Phase 4: Advanced ML & Statistical Learning Theory

Duration: 8–12 weeks

Difficulty: Hard to Research-Level

Goal: Bridge the gap between practitioner and researcher. Understand *why* algorithms work, their theoretical guarantees, and cutting-edge classical ML methods.

5.1 Statistical Learning Theory

Duration: 3–4 weeks

1. PAC Learning Framework

- Probably Approximately Correct (PAC) model
- Sample complexity bounds
- Realisable vs. agnostic PAC learning
- Computational learning theory basics

2. VC Theory

- VC dimension: definition, computation for linear classifiers, decision trees
- VC generalisation bound: $R(h) \leq \hat{R}(h) + \sqrt{\frac{d_{VC} \log(2N/d_{VC}) + \log(4/\delta)}{N}}$
- Structural Risk Minimisation
- Growth function and Sauer's lemma

3. Rademacher Complexity

- Definition and relation to generalisation
- Rademacher bounds for linear models, kernel methods
- Comparison with VC bounds

4. Regularisation Theory

- Tikhonov regularisation, reproducing kernel Hilbert spaces (RKHS)
- Representer theorem (full statement and proof sketch)
- Stability-based generalisation bounds

5. Online Learning & Regret Bounds

- Online convex optimisation framework
- Regret: $R_T = \sum_{t=1}^T f_t(\mathbf{x}_t) - \min_{\mathbf{x}} \sum_{t=1}^T f_t(\mathbf{x})$
- Online Gradient Descent: $O(\sqrt{T})$ regret
- Multiplicative Weights / Hedge algorithm
- Bandit algorithms: UCB, Thompson Sampling
- Expert advice and the Weighted Majority algorithm

5.2 Bayesian Machine Learning (Advanced)

Duration: 2–3 weeks

1. Gaussian Processes (GP)

- GP as a distribution over functions: $f(\mathbf{x}) \sim \mathcal{GP}(m(\mathbf{x}), k(\mathbf{x}, \mathbf{x}'))$
- GP regression: posterior mean and variance, predictive uncertainty
- Kernel design: composing kernels, automatic relevance determination (ARD)
- Hyperparameter learning via marginal likelihood optimisation
- Sparse GP approximations: inducing points, FITC, variational approach
- GP classification (Laplace approximation, EP)

2. Bayesian Nonparametrics

- Dirichlet Process (DP) and DP mixtures
- Chinese Restaurant Process, stick-breaking construction
- Indian Buffet Process (overview)

3. Advanced Approximate Inference

- Variational inference: ELBO derivation, mean-field, structured VI
- MCMC: Hamiltonian Monte Carlo, NUTS
- Expectation Propagation
- Amortised inference (connection to VAEs — covered in DL volume)

5.3 Graphical Models

Duration: 2–3 weeks

1. Directed Graphical Models (Bayesian Networks)

- DAGs, conditional independence, d-separation
- Factorisation: $p(\mathbf{x}) = \prod_i p(x_i | \text{pa}(x_i))$
- Structure learning: constraint-based (PC algorithm), score-based (BIC, BDeu)
- Parameter estimation: MLE, Bayesian
- Exact inference: variable elimination, junction tree algorithm

2. Undirected Graphical Models (Markov Random Fields)

- Clique potentials, partition function Z
- Hammersley-Clifford theorem
- Conditional Random Fields (CRFs) for structured prediction

3. Approximate Inference in Graphical Models

- Loopy belief propagation

- Variational message passing

4. Hidden Markov Models (HMMs)

- Forward-backward algorithm
- Viterbi algorithm (MAP sequence)
- Baum-Welch (EM for HMMs)

5.4 Advanced Topics in Classical ML

Duration: 2–3 weeks

1. Kernel Methods (Advanced)

- Kernel PCA, kernel k-means, kernel CCA
- String kernels, graph kernels
- Multiple Kernel Learning (MKL)
- Random features / Nyström approximation for scalable kernels

2. Semi-Supervised Learning

- Self-training, co-training
- Label propagation on graphs
- Transductive SVM
- Assumptions: smoothness, cluster, manifold

3. Active Learning

- Uncertainty sampling, query-by-committee
- Expected model change, expected error reduction
- Bayesian active learning: BALD

4. Multi-Task & Transfer Learning (Classical)

- Shared representations, task relatedness
- Multi-task regularisation
- Domain adaptation: covariate shift, importance weighting

5. Causal Inference (Introduction)

- Correlation vs. causation; Simpson's paradox
- Structural Causal Models (SCMs), do-calculus
- Potential outcomes framework (Rubin)
- Average Treatment Effect (ATE), propensity score matching
- Instrumental variables, difference-in-differences

6. Fairness, Accountability & Transparency

- Fairness definitions: demographic parity, equalised odds, calibration
- Impossibility theorems (Chouldechova, Kleinberg et al.)
- Bias mitigation: pre-processing, in-processing, post-processing
- Interpretability: SHAP values, LIME, partial dependence plots

Resources & References

- **Learning Theory:** *Foundations of Machine Learning* — Mohri, Rostamizadeh, Talwalkar; *Understanding Machine Learning* — Shalev-Shwartz & Ben-David (free PDF)
- **Gaussian Processes:** *Gaussian Processes for Machine Learning* — Rasmussen & Williams (free PDF)
- **Graphical Models:** *Probabilistic Graphical Models* — Koller & Friedman; or Jordan's course notes
- **Causal Inference:** *Causal Inference in Statistics: A Primer* — Pearl, Glymour, Jewell; *The Book of Why* — Pearl & Mackenzie
- **Fairness:** Barocas, Hardt, Narayanan — *Fairness and Machine Learning* (free online)
- **Courses:** Stanford CS229T (Statistical Learning Theory); CMU 10-708 (Probabilistic Graphical Models)

6 Phase 5: ML Systems, Engineering & Scalability

Duration: 4–6 weeks

Difficulty: Moderate

Goal: Be able to take an ML algorithm from prototype to production. Understand the systems-level challenges of deploying ML at scale.

6.1 Data Engineering for ML

1. Data Collection & Quality

- Data wrangling with pandas, Polars
- Handling missing data: imputation strategies (mean, KNN, MICE, missForest)
- Outlier detection and treatment
- Data versioning: DVC, LakeFS

2. Feature Engineering

- Numerical: scaling (standard, min-max, robust), log/Box-Cox transforms, binning
- Categorical: one-hot, ordinal, target encoding, frequency encoding
- Text: TF-IDF, BM25, bag-of-words, n-grams
- Time series: lag features, rolling statistics, Fourier features, calendar features
- Automated: Featuretools, TPOT

3. Feature Selection

- Filter methods: mutual information, ANOVA F-test, χ^2
- Wrapper methods: forward/backward selection, RFE
- Embedded methods: L1 regularisation, tree-based importance
- Boruta algorithm

6.2 ML Pipeline Design

1. Experiment Tracking

- MLflow, Weights & Biases, Neptune
- Logging metrics, parameters, artefacts
- Reproducibility: random seeds, environment pinning

2. Model Serialisation & Serving

- Pickle, joblib, ONNX
- REST APIs: FastAPI, Flask
- Batch vs. online prediction
- Model registries

3. Monitoring & Maintenance

- Data drift detection: PSI, KS test, MMD
- Concept drift: DDM, ADWIN
- Model retraining strategies
- A/B testing for ML models

6.3 Scalable ML

1. Computational Considerations

- Algorithmic complexity: training and inference
- Sparse data structures and algorithms
- Approximate methods: random projections (Johnson-Lindenstrauss), locality-sensitive hashing

2. Distributed & Parallel ML

- Data parallelism vs. model parallelism
- MapReduce paradigm
- Spark MLlib, Dask-ML
- Parameter servers (overview)

Hands-On Projects

1. **End-to-End Pipeline:** Build a complete ML pipeline: data ingestion → feature engineering → model training → hyperparameter tuning → evaluation → REST API serving → monitoring dashboard.
2. **Drift Detection System:** Implement a monitoring system that detects data/concept drift on a streaming dataset and triggers retraining.

Resources & References

- **Primary:** *Designing Machine Learning Systems* — Chip Huyen
- **Engineering:** *Machine Learning Engineering* — Burkov
- **Practice:** *Building Machine Learning Pipelines* — Hapke & Nelson (O'Reilly)
- **Paper:** Sculley et al., “Hidden Technical Debt in ML Systems,” NeurIPS 2015

7 Phase 6: Research Methodology & Paper Writing

Duration: 6–10 weeks

Difficulty: Research-Level

Goal: Transition from consumer of ML knowledge to producer. Learn to read, critique, replicate, and write ML research papers.

7.1 Reading Research Papers

1. The Three-Pass Method (Keshav, 2007)

- **Pass 1** (5–10 min): Title, abstract, introduction, section headings, conclusions. Identify the category, context, correctness, contributions, clarity.
- **Pass 2** (1 hr): Read with care, ignoring proofs. Note key figures, mark unread references.
- **Pass 3** (4–5 hrs): Virtually re-implement the paper. Identify assumptions, missing details, and potential improvements.

2. Building a Literature Map

- Use Connected Papers, Semantic Scholar, Google Scholar
- Track papers in a reference manager (Zotero, Mendeley, Paperpile)
- Maintain a reading log with 1-paragraph summaries

3. Essential Papers to Read (Classical ML)

- Vapnik (1995) — *The Nature of Statistical Learning Theory*
- Breiman (2001) — *Random Forests*
- Friedman (2001) — *Greedy Function Approximation: A Gradient Boosting Machine*
- Rasmussen & Williams (2006) — GP chapter introductions
- Chen & Guestrin (2016) — *XGBoost*
- Lundberg & Lee (2017) — *SHAP values*
- Pearl (2009) — *Causality* (selected chapters)

7.2 Designing Experiments

1. Experimental Methodology

- Formulating a clear research question / hypothesis
- Choosing appropriate baselines
- Controlling for confounders: random seeds, hardware, data splits
- Ablation studies: systematically removing components
- Statistical significance: paired t-tests, Wilcoxon tests, bootstrap confidence intervals

2. Benchmark Datasets

- UCI ML Repository, OpenML
- Kaggle Datasets
- Domain-specific: ImageNet, GLUE/SuperGLUE (NLP), LibSVM
- Creating your own benchmark: data collection, annotation, quality assurance

3. Reproducibility

- Code: version control (Git), containerisation (Docker), environment files
- Data: checksums, data statements, datasheets for datasets
- Experiments: config files, seed logging, result caching
- Reproducibility checklists (ML Reproducibility Challenge)

7.3 Writing ML Papers

1. Paper Structure

- **Title:** Concise, specific, searchable
- **Abstract:** Problem, method, result, significance (150–250 words)
- **Introduction:** Motivation, gap in literature, contributions (bulleted list), paper outline
- **Related Work:** Organised by theme, not chronologically; position your work
- **Method:** Mathematical formulation, algorithm pseudocode, complexity analysis
- **Experiments:** Setup (datasets, baselines, metrics, hyperparameters), results tables/figures, ablation studies
- **Discussion:** Interpretation, limitations, failure cases
- **Conclusion:** Summary, future work

2. Writing Quality

- Use $\text{\LaTeX}$ with conference templates (NeurIPS, ICML, AISTATS)
- Figures: matplotlib/seaborn with publication-quality formatting
- Tables: proper use of `booktabs`, consistent significant figures
- Mathematical notation: consistent, defined on first use
- Active voice, present tense for contributions

3. Peer Review Process

- Top venues: NeurIPS, ICML, ICLR, AISTATS, UAI, JMLR, TMLR
- Understanding reviewer criteria: novelty, significance, clarity, correctness
- Writing rebuttals: be factual, add experiments, stay professional
- Reviewing papers yourself (volunteer for workshops)

7.4 Research Project Execution

1. Finding Research Problems

- Read recent workshop papers and “future work” sections
- Identify limitations in existing methods
- Apply methods from one domain to another
- Scale up theoretical results; or find theory for empirical results

2. Capstone Research Project

- Choose a problem at the intersection of your interests
- Write a 2-page proposal: motivation, related work, proposed method, expected experiments
- Execute: implement, experiment, iterate
- Write up as a full conference-format paper (8+ pages)
- Present to peers, collect feedback, revise

Hands-On Projects

1. **Paper Replication:** Pick a top-venue paper (e.g., XGBoost, SHAP). Reproduce its key experiments. Write a 4-page report on what matched, what didn't, and why.
2. **Novel Research Paper:** Identify a gap in classical ML, propose a method, run experiments with proper baselines, write it as a workshop paper (4 pages).
3. **Survey Paper:** Write a 10-page survey on a focused topic (e.g., “Ensemble Methods for Tabular Data: A Unified View”).

Resources & References

- **Paper Reading:** Keshav, “How to Read a Paper” (2007)
- **Writing:** Simon Peyton Jones, “How to Write a Great Research Paper” (talk); Whitesides, “Writing a Scientific Paper”
- **LaTeX:** Overleaf tutorials; NeurIPS/ICML L^AT_EX templates
- **Research Process:** Richard Hamming, “You and Your Research” (1986 talk)
- **Communities:** r/MachineLearning, Papers With Code, ML Twitter/X, local ML meetups

8 Appendices

8.1 Appendix A: Recommended Book Progression

#	Book	Authors	Phase
1	Mathematics for Machine Learning	Deisenroth, Faisal, Ong	Phase 1–2
2	An Introduction to Statistical Learning (ISL)	James, Witten, Hastie, Tibshirani	Phase 3
3	Pattern Recognition and Machine Learning	Bishop	Phase 2–4
4	The Elements of Statistical Learning	Hastie, Tibshirani, Friedman	Phase 3–4
5	Convex Optimization	Boyd & Vandenberghe	Phase 1
6	Gaussian Processes for ML	Rasmussen & Williams	Phase 4
7	Understanding Machine Learning	Shalev-Shwartz & Ben-David	Phase 4
8	Probabilistic Graphical Models	Koller & Friedman	Phase 4
9	Designing ML Systems	Chip Huyen	Phase 5
10	Causality	Judea Pearl	Phase 4

8.2 Appendix B: Weekly Study Template

Monday–Tuesday: Theory & reading (textbook chapters, lecture videos)
Wednesday: Mathematical exercises (proofs, derivations)
Thursday–Friday: Coding implementation (from scratch, then with libraries)
Saturday: Paper reading (1–2 papers), experiment with ideas
Sunday: Review, update learning journal, plan next week
Target: 15–20 hours per week (adjust to your schedule)

8.3 Appendix C: Python Environment Setup

```
# Create conda environment
conda create -n ml_research python=3.11 -y
conda activate ml_research

# Core scientific stack
pip install numpy scipy pandas matplotlib seaborn

# ML libraries
pip install scikit-learn xgboost lightgbm catboost

# Bayesian / Probabilistic
pip install pymc arviz gpytorch botorch

# Experiment tracking
pip install mlflow optuna wandb

# Visualisation & notebooks
pip install jupyter plotly

# Development
pip install pytest black ruff
```

8.4 Appendix D: Key Mathematical Notation

Notation	Meaning
$\mathbf{x} \in \mathbb{R}^d$	Feature vector (column vector)
$\mathbf{X} \in \mathbb{R}^{N \times d}$	Data matrix (N samples, d features)
$y, \mathbf{y}$	Target variable (scalar, vector)
$\boldsymbol{\theta}, \mathbf{w}$	Model parameters / weights
$\mathcal{L}(\cdot)$	Loss function
∇f	Gradient of f
$\mathbf{H}$	Hessian matrix
$\ \cdot\ _p$	ℓ_p norm
$\mathbb{E}[\cdot]$	Expectation
$\text{Var}(\cdot)$	Variance
$p(\cdot), P(\cdot)$	Probability density / mass function
$\mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma})$	Multivariate Gaussian distribution
$D_{\text{KL}}(\cdot\ \cdot)$	KL divergence
$\mathcal{O}(\cdot)$	Big-O complexity
$\mathbf{I}_d$	$d \times d$ identity matrix
$\mathbf{A} \succ 0$	Positive definite matrix

What's Next?

After completing this plan, you will have a solid research-level foundation in classical Machine Learning. **Volume II** will cover Deep Learning & Neural Networks: MLPs, CNNs, RNNs/LSTMs, Transformers, GANs, Diffusion Models, Reinforcement Learning, and frontier research.